UNIVERSIDADE REGIONAL DE BLUMENAU

CENTRO DE CIÊNCIAS EXATAS E NATURAIS

DEPARTAMENTO DE SISTEMAS E COMPUTAÇÃO

DISCIPLINA: SISTEMAS DISTRIBUÍDOS

FELIPE GROCHOVSKI

ELCIO CLEITON WIPPEL

MICROCHEFS: SISTEMA DE MENSAGERIA COM RABBITMQ

Trabalho Prático – Mensageria

BLUMENAU/SC

2025

SUMÁRIO

1 DESCRIÇÃO DO CENÁRIO 3
1.1 Contextualização 3
1.2 Justificativa da Comunicação Assíncrona via Mensageria 3
2 ARQUITETURA DA SOLUÇÃO 5
2.1 Componentes Produtores e Consumidores 5
2.1.1 Produtores de Mensagens 5
2.1.2 Consumidores de Mensagens 6
2.2 Fluxo de Mensagens 6
2.3 Diagrama da Arquitetura de Mensageria 7
2.4 Estratégias de Escalabilidade, Confiabilidade e Tolerância a Falhas 9
2.4.1 Escalabilidade 9
2.4.2 Confiabilidade e Tolerância a Falhas 9
3 CONFIGURAÇÃO DO RABBITMQ 11
3.1 Exchanges, Filas e Bindings 11
3.2 Políticas de Retry e Dead-Letter-Queue 11
3.3 Requisitos de Segurança 12
4 EXEMPLOS DE USO 13
4.1 Caso de Uso 1: Pedido Pago e Enviado à Cozinha 13
4.2 Caso de Uso 2: Atualização de Status pelo Preparo 13
4.3 Caso de Uso 3: Tratamento de Mensagem com Erro 14
5 CONSIDERAÇÕES TÉCNICAS 15
5.1 Tecnologias e Linguagens Utilizadas 15
5.2 Padrão de Mensagens 15
5.3 Boas Práticas Adotadas 16
REFERÊNCIAS 18

1 DESCRIÇÃO DO CENÁRIO

1.1 Contextualização

O cenário trabalhado neste projeto é denominado MicroChefs, uma plataforma de gerenciamento de pedidos para restaurantes baseada em arquitetura de microsserviços. O sistema possui como microsserviços principais o Pedido-Service e o User-Service, além dos serviços auxiliares de infraestrutura e suporte: Cozinha-Service e DLQSupport.

A plataforma simula o ciclo completo de um pedido em restaurante: da criação, passando pelo pagamento, preparo na cozinha, até a entrega final ao cliente.

1.2 Justificativa da Comunicação Assíncrona via Mensageria

A comunicação assíncrona via mensageria com RabbitMQ foi adotada por uma razão fundamental: o desacoplamento entre os serviços e a resiliência do sistema como um todo. Em um ambiente de alta demanda, como uma cozinha de restaurante, o serviço de pedidos não deve ser bloqueado aguardando o processamento do preparo.

Com a mensageria, o cliente recebe confirmações imediatas de suas ações, enquanto o fluxo de produção e as atualizações de status ocorrem de forma assíncrona, garantindo que o sistema suporte picos de carga sem perda de dados. Adicionalmente, o uso de filas evita o excesso desnecessário de chamadas HTTP via endpoints, possibilitando um envio simples e eficiente de mensagens. O sistema opera de forma "universal": se futuramente um novo serviço

precisar consumir atualizações de pedidos, basta implementar o serviço com a fila e a binding key correspondente, sem necessidade de alterações nos demais componentes.

2 ARQUITETURA DA SOLUÇÃO

2.1 Componentes Produtores e Consumidores

A arquitetura utiliza o RabbitMQ como mediador (Broker) central para a troca de informações entre os serviços. A seguir, são detalhados os produtores e consumidores de mensagens do sistema:

2.1.1 Produtores de Mensagens

- Pedido Producer (Pedido-Service): responsável por enviar pedidos pagos para o preparo na cozinha, repassar as atualizações recebidas da cozinha para os demais serviços e encaminhar mensagens problemáticas para a Dead-Letter-Queue (DLQ).
- Pedido Producer (Cozinha): responsável por enviar atualizações de status do pedido para o Pedido-Service, informando cada estado até a finalização do preparo, além de enviar mensagens problemáticas para a DLQ em casos de falhas.
- MensagemProducer (DLQSupport): responsável pelo reenvio de mensagens corrigidas para as devidas filas do RabbitMQ, adaptando o formato de entrega com base na fila de origem da mensagem.
- User Producer (User-Service): responsável por receber notificações de atualização de status enviadas pelo Pedido-Service e repassá-las ao User-Service, mantendo o cliente informado sobre o andamento do pedido.

2.1.2 Consumidores de Mensagens

- Update Pedido Consumer (Pedido-Service): responsável por consumir as atualizações de pedido enviadas pela cozinha, capturando exceções na leitura e manipulação dos dados para o controle de mensagens.
- Pedido Consumer (Cozinha): responsável por consumir o pedido enviado pelo Pedido-Service, coletando exceções que possam indicar mensagens destinadas à Dead-Letter-Queue.
- Support Consumer (DLQSupport): responsável por consumir as mensagens problemáticas e encaminhá-las ao sistema para que o suporte realize a devida correção via Editor JSON.
- User Consumer (User-Service): responsável por consumir as notificações de status do pedido, atualizando o serviço de usuários em tempo real para manter o cliente informado.

2.2 Fluxo de Mensagens

O fluxo de mensagens utiliza uma Topic Exchange denominada pedido-exchange para permitir roteamento flexível entre os serviços. Abaixo são descritas as etapas do fluxo completo:

- **Criação do Pedido:** Após ser criado, o pedido aguarda confirmação de pagamento ou cancelamento. Pedidos pagos em dinheiro são automaticamente processados como pagos. Quando confirmado, o Pedido-Service publica na pedido-exchange com a routing key pedido-key.pago, convertendo a DTO para JSON que trafegará na cozinha-queue.

- **Atualização de Status:** A cozinha recebe a mensagem e inicia a produção, simulando atualizações a cada 15 segundos. A cada mudança, publica na mesma exchange com a key pedido-key.update, convertendo a DTO em JSON para tráfego na pedido-queue.
- **Notificação ao Cliente:** O Pedido-Service emite alertas a cada novo estado do pedido para a pedido-exchange, utilizando a key pedido-key.status, direcionando as mensagens para a fila user-queue.
- **Tolerância a Falhas:** Todos os consumers podem acionar seus producers para encaminhar mensagens problemáticas à DLQ. Falhas de infraestrutura (banco, RabbitMQ) ativam a política de retry automático. Erros de dados (JSON mal formatado ou campos inválidos) são enviados imediatamente à dead-letter-exchange com a key dead-message, trafegando na dead-letter-queue para processamento pelo serviço de suporte.
- **Reprocessamento Manual:** O DLQSupport consome as mensagens da dead-letter-queue e disponibiliza um Editor JSON para que o suporte realize a correção. Após a correção, o MensagemProducer reenvia a mensagem para sua fila original (cozinha-queue, pedido-queue ou user-queue).

2.3 Diagrama da Arquitetura de Mensageria

A Figura 1 apresenta o diagrama do sistema de mensageria capturado do painel do RabbitMQ, evidenciando as exchanges, filas e routing keys utilizadas na solução, bem como os producers e consumers associados a cada componente.

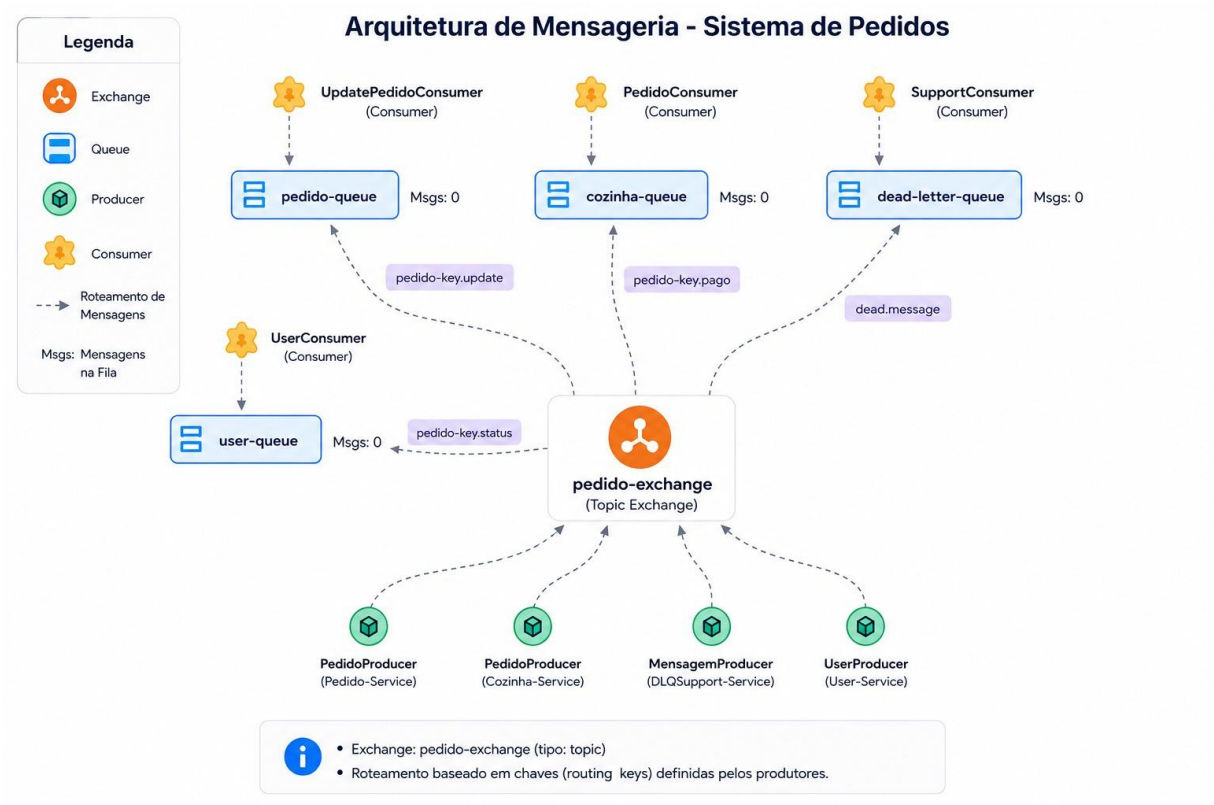

Figura 1 – Diagrama de mensageria RabbitMQ do sistema MicroChefs

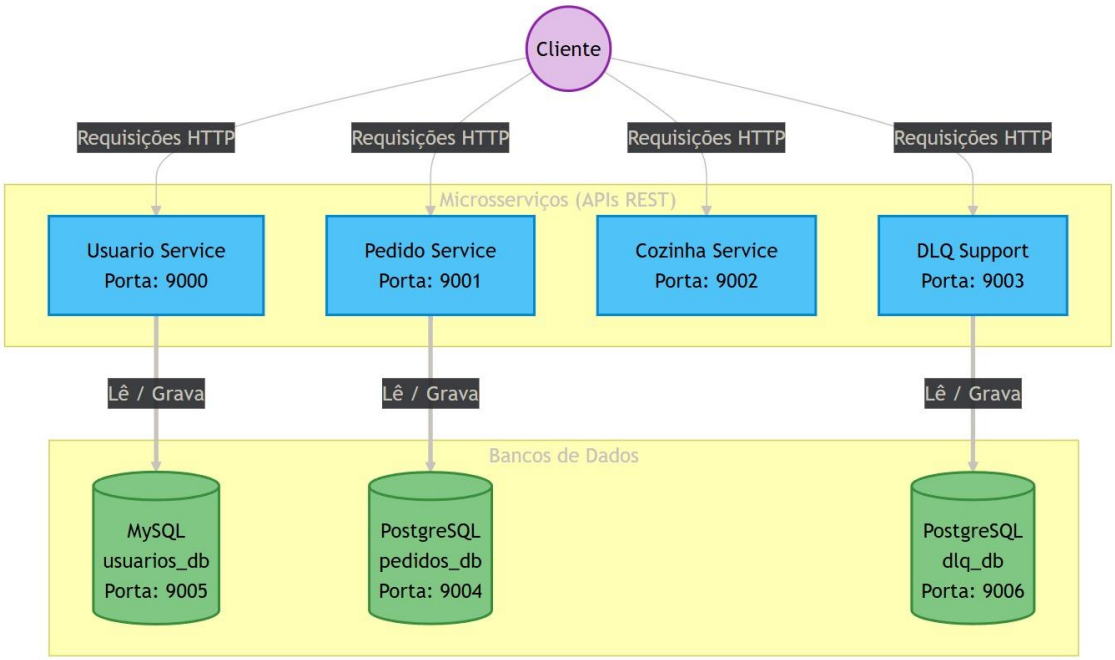

Figura 2 – Diagrama de arquitetura do sistema MicroChefs

2.4 Estratégias de Escalabilidade, Confiabilidade e Tolerância a Falhas

2.4.1 Escalabilidade

O foco da escalabilidade está no serviço de Cozinha, por ser o componente mais suscetível à sobrecarga de mensagens em cenários de alta demanda. Como proposta futura, foi planejada a utilização da biblioteca Prometheus para monitoramento de métricas da aplicação, como quantidade de mensagens na fila e tempo de processamento por instância, permitindo uma estratégia mais automatizada de escalabilidade.

Entretanto, na versão apresentada do sistema, o monitoramento automatizado ainda não foi implementado. No momento, a arquitetura permite a criação dinâmica de novas instâncias do serviço de cozinha por meio do Docker Compose, conforme a necessidade. O balanceamento de carga entre as instâncias ocorre automaticamente pelo RabbitMQ, que distribui as mensagens da cozinha-queue entre os consumers disponíveis utilizando a estratégia round-robin.

2.4.2 Confiabilidade e Tolerância a Falhas

Para garantir a confiabilidade do sistema, foram implementadas as seguintes estratégias:

- Política de Retry: aplicada para falhas temporárias de infraestrutura (banco de dados, RabbitMQ), garantindo o reenvio automático da mensagem para revalidação.

- Dead-Letter-Queue (DLQ): camada adicional de segurança para mensagens que falham mesmo após as tentativas de retry. O serviço DLQSupport consome automaticamente as mensagens da DLQ e disponibiliza interface para reprocessamento manual.
- Acknowledge Manual (ACK): a confirmação de recebimento é realizada manualmente pelos consumers, garantindo que a mensagem seja removida da fila somente após o processamento bem-sucedido.
- Idempotência: os consumers verificam se o identificador do pedido já foi processado, evitando duplicidade de operações em caso de reentrega de mensagens.

3 CONFIGURAÇÃO DO RABBITMQ

3.1 Exchanges, Filas e Bindings

A configuração do RabbitMQ foi estruturada para suportar o roteamento flexível de mensagens entre os microsserviços. Abaixo são descritos os principais componentes configurados:

Exchange	Tipo	Finalidade
pedido-exchange	Topic	Roteamento central de todas as mensagens de pedido
dead-letter-exchange	Direct	Direcionamento de mensagens problemáticas para a DLQ

Quadro 1 – Exchanges configuradas no RabbitMQ

Fila	Routing Key	Consumidor
cozinha-queue	pedido-key.pago	Cozinha-Service
pedido-queue	pedido-key.update	Pedido-Service (UpdateConsumer)
user-queue	pedido-key.status	User-Service
dead-letter-queue	dead-message	DLQSupport (SupportConsumer)

Quadro 2 – Filas, routing keys e consumidores do sistema MicroChefs

3.2 Políticas de Retry e Dead-Letter-Queue

O sistema implementa duas estratégias distintas para o tratamento de falhas de mensagens, diferenciando erros de infraestrutura de erros de dados:

- Erros de Infraestrutura (INFRA): falhas relacionadas ao banco de dados ou ao próprio RabbitMQ são tratadas com política de retry automático, tentando o reenvio da mensagem por um número configurável de vezes antes de encaminhá-la à DLQ.
- Erros de Dados (DATA): mensagens com JSON mal formatado ou campos inválidos são encaminhadas imediatamente para a dead-letter-queue, sem tentativas de retry, pois uma nova tentativa resultaria no mesmo erro.

3.3 Requisitos de Segurança

A autenticação dos usuários é realizada por meio de JSON Web Token (JWT). Após efetuar o login, o usuário recebe um token assinado que contém suas informações de autenticação e autorização. Esse token é enviado nas requisições subsequentes e validado pelos serviços da aplicação antes do acesso aos recursos protegidos.

4 EXEMPLOS DE USO

4.1 Caso de Uso 1: Pedido Pago e Enviado à Cozinha

Entrada: O cliente finaliza um pedido e efetua o pagamento (ou paga em dinheiro, resultando em confirmação automática).

Processamento: O Pedido-Service detecta o status "pago" e aciona o PedidoProducer, que publica uma mensagem na pedido-exchange com a routing key pedido-key.pago. A mensagem é convertida de DTO para JSON e encaminhada para a cozinha-queue.

Saída esperada: O PedidoConsumer da Cozinha consome a mensagem da cozinha-queue e inicia o processo de preparo do pedido.

4.2 Caso de Uso 2: Atualização de Status pelo Preparo

Entrada: A cada 15 segundos (simulado), a cozinha avança o estado do pedido (ex.: "Em preparo", "Pronto para entrega").

Processamento: O PedidoProducer da Cozinha publica na pedido-exchange com a key pedido-key.update, convertendo o DTO em JSON para tráfego na pedido-queue. O UpdatePedidoConsumer do Pedido-Service consome a mensagem e atualiza o banco de dados.

Saída esperada: O Pedido-Service emite novas notificações com a key pedido-key.status, que são consumidas pela fila user-queue, mantendo o cliente informado sobre o andamento do pedido.

4.3 Caso de Uso 3: Tratamento de Mensagem com Erro

Entrada: Um consumer recebe uma mensagem com JSON inválido ou campo obrigatório ausente.

Processamento: O consumer identifica o erro de dados e aciona imediatamente o producer correspondente para publicar na dead-letter-exchange com a key dead-message. A mensagem, junto com o log de erro completo (em formato JSON), é encaminhada para a dead-letter-queue e consumida pelo SupportConsumer do DLQSupport.

Saída esperada: O Editor JSON do DLQSupport exibe a mensagem para análise e correção manual pelo suporte técnico. Após a correção, o MensagemProducer reenvia a mensagem para sua fila original.

5 CONSIDERAÇÕES TÉCNICAS

5.1 Tecnologias e Linguagens Utilizadas

O projeto MicroChefs foi desenvolvido utilizando um conjunto de tecnologias, evidenciando a capacidade da arquitetura de microsserviços em integrar stacks diferentes:

- Java com Spring Boot: utilizado no desenvolvimento do Pedido-Service, incluindo as bibliotecas Spring AMQP para integração com RabbitMQ e Spring Data JPA para persistência.
- C# com ASP.NET: utilizado nos microsserviços auxiliares, integrando-se ao RabbitMQ via client oficial .NET.
- HTML, CSS, JS: utilizado para os serviços de suporte (DLQSupport), aproveitando a natureza assíncrona da plataforma para consumo eficiente de mensagens.
- PostgreSQL e MySQL: bancos de dados relacionais utilizados para persistência dos dados dos diferentes microsserviços.
- RabbitMQ: broker de mensageria responsável pela comunicação assíncrona entre todos os componentes.
- Prometheus: biblioteca de monitoramento utilizada para coleta de métricas de performance, especialmente do serviço de cozinha.
- Docker e Docker Compose: utilizados para containerização de todos os serviços e orquestração dos containers, incluindo escalabilidade horizontal da cozinha.
- Postman: utilizado para testes dos endpoints HTTP dos microsserviços.

5.2 Padrão de Mensagens

Todos os serviços do sistema MicroChefs adotam o padrão JSON para a serialização e deserialização de mensagens que trafegam pelo RabbitMQ. A escolha por JSON justifica-se pela sua ampla compatibilidade entre as diferentes linguagens utilizadas no projeto (Java, C# e NodeJS), pela legibilidade humana que facilita a depuração e pela facilidade de validação da estrutura das mensagens.

Cada mensagem carrega uma DTO (Data Transfer Object) convertida para JSON, contendo os campos necessários para o processamento pelo consumer de destino. As mensagens direcionadas à Dead-Letter-Queue incluem adicionalmente o log completo do erro, com informações como fila de origem, conteúdo original da mensagem e descrição da falha.

5.3 Boas Práticas Adotadas

As seguintes boas práticas foram implementadas para garantir desempenho, integridade e confiabilidade na troca de mensagens:

- Idempotência: os consumers verificam se o identificador único do pedido já foi processado anteriormente, evitando a duplicidade de operações em casos de reentrega de mensagens pelo mecanismo de retry.
- Acknowledge Manual (ACK): a confirmação de recebimento de mensagem é realizada manualmente após o processamento bem-sucedido. Isso garante que a mensagem seja removida da fila apenas quando o processamento for concluído com êxito, prevenindo perda de dados.

- Separação de erros por categoria: a distinção entre erros de infraestrutura (sujeitos a retry) e erros de dados (encaminhados imediatamente à DLQ) evita tentativas desnecessárias de reprocessamento de mensagens estruturalmente inválidas.
- Serialização padronizada em JSON: o uso consistente de JSON como padrão de mensagens garante interoperabilidade entre serviços desenvolvidos em linguagens distintas.
- Variáveis de ambiente para credenciais: as credenciais do RabbitMQ e dos bancos de dados são gerenciadas exclusivamente via variáveis de ambiente no Docker Compose, nunca sendo incluídas no código-fonte.
- Binding key universal: a adoção da routing key pedido-key.status como chave universal de notificação de status permite que novos serviços se integrem ao sistema sem necessidade de alterações nos componentes existentes.
- Containerização completa com Docker: todos os serviços, incluindo o RabbitMQ e os bancos de dados, são executados em containers, garantindo reprodutibilidade do ambiente e facilitando a escalabilidade.

REFERÊNCIAS

RABBITMQ. RabbitMQ Documentation. Disponível em: <https://www.rabbitmq.com/documentation.html>. Acesso em: maio 2025.

SPRING. Spring AMQP Reference Documentation. Disponível em: <https://docs.spring.io/spring-amqp/docs/current/reference/html/>. Acesso em: maio 2025.

PROMETHEUS. Prometheus Documentation. Disponível em: <https://prometheus.io/docs/>. Acesso em: maio 2025.

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: maio 2025.

NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 2. ed. Sebastopol: O'Reilly Media, 2021.